Developing and Debugging Activity Book

Data Modeling for Qlik Sense

© 2023 QlikTech International AB. All rights reserved. All company and/or product names may be trade names, trademarks and/or registered trademarks of the respective owners with which they are associated.

DMQS May 2023

Table of Contents

Developing and Debugging Activity Book

Table of Contents

Activity 1: Use variables

Challenge 2
Hints 3
Solution 5

Activity 2: Add an Include statement

Challenge 5
Hints 6
Solution 8

Activity 3: Use a For loop to load all sheets

Challenge 8
Hints 9
Solution 11

Activity 4: Debugging a data load script

Challenge 12
Hints 12

Activity 1: Use variables

Summary:

- Create and use variables for centrally managing data connection usage.
- Create and display variables for calculating data model reload time.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, add variables that will help you maintain important load information by completing the following tasks:

- Add new variables in the **Main** script section, after the system variables, as follows:

```
LET vLoadStart = Now();
SET vPathFiles = 'lib://DataFiles/';
SET vUseDBConnections = 1;
```

Qlik Sense on Windows: Multiple data connections have been used and will need to be catered for, instead of the generic **DataFiles**:

```
SET vPathExcel = 'lib://ExcelFiles (qse_student)/';
SET vPathText = 'lib://TextFiles (qse_student)/';
SET vPathXML = 'lib://XMLFiles (qse_student)/';
```

- At the end of the **LoadComplete** section, add the following variable:
  `LET vLoadComplete = Now();`
- On the **Main**, **MappingTables**, **Orders**, **Products**, **Customers** and **Shipments** script sections, wrap the **LIB CONNECT TO** statement within an **IF** conditional statement and add a **TRACE** line, as follows:

```
IF vUseDBConnections = 1 THEN
TRACE ---- CONNECT TO ABC ----;
LIB CONNECT TO '<connection name>';
END IF;
```

- Replace the data connection `lib:// ` statements for file sources with the relevant variables, using the **Search and replace** feature.
- Test a reload with the **vUseDBConnections** set to 1. Change it to 0 and reload again.
- Add a new app sheet named *Variables* and create the following **Text & image** objects:
  Load Start: `='Load Start: ' & vLoadStart`
  Load Complete: `='Load Complete: ' & vLoadComplete`
  Reload Time: `='Reload time: ' & ReloadTime()`

Hints

- The **Search and replace** feature will easily replace the data connection statements with the new variables.

Step-by-step guide

1. Near the bottom of the **Main** script section, just before the **LIB CONNECT** statement, create new variables by adding the following text:

```
// **** Set Variables ****
LET vLoadStart = Now();
SET vPathFiles = 'lib://DataFiles/';
SET vUseDBConnections = 1;
```

Copy to clipboard

- The variable **vLoadStart** is equal to the result from the **Now F** function, which returns the current time from the system clock. Used in this script, the variable **vLoadStart** contains the time the data load started.
- The **vPath** variable can be used as references to the data connections.

Qlik Sense on Windows: Multiple data connections have been used. Therefore, the script will need to be edited accordingly:

```
// **** Set variables ****
LET vLoadStart = Now();
SET vPathText = 'lib://TextFiles (qse_student)/';
SET vPathExcel = 'lib://ExcelFiles (qse_student)/';
SET vPathXML = 'lib://XMLFiles (qse_student)/';
SET vUseDBConnections = 1;
```

2. Navigate to the end of the **LoadComplete** section, and add the following line:

```
// **** Set LoadComplete Variable ****
LET vLoadComplete = Now();
```

Copy to clipboard

This lets the variable vLoadComplete equal the result of the NOW function, but this is a later time than the time set in vLoadStart.

3. Use the **Search and replace** feature to replace all the data connection `lib:// ` statements in your script with the corresponding variable when applicable.
For instance, for the **DataFiles** connection, search for *lib://DataFiles/* and replace it with *$(vPathFiles)*. Replace all the items found **except** the ones in the **Main** section.

> **Note:** If you use any external scripts, make sure you also change the data connection inside the file.

4. On the **Main**, **MappingTables**, **Orders**, **Products**, **Customers** and **Shipments** script sections, the LIB CONNECT line should be wrapped in a conditional script statement so that it is only executed if the vUseDBConnections variable equals one. Use the following script:

```
 // **** Conditional connection ****

IF vUseDBConnections = 1 THEN
   TRACE ---- CONNECT TO ABC ----;
    LIB CONNECT TO'ABC';
END IF;
```

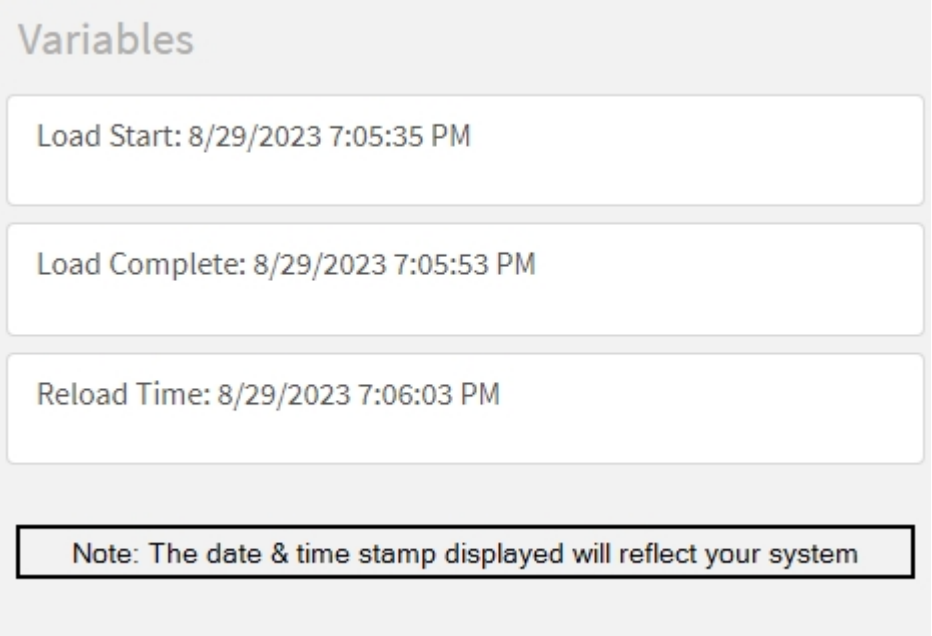
Copy to clipboard

5. Load the data. Observe within the **Data load progress** dialog, the connection line is preceded by the trace output from the script.
6. Using **Analyze** mode, create a new sheet named *Variables*.
7. Enable the edit mode. Validate **vLoadStart** and **vLoadComplete** have been populated in the **Variables** list.
8. Add a **Text & image** object and set the size to *6* columns wide and *2* rows high.
9. Add two more **Text & image** objects. Position them underneath each other and resize them the same.
10. From the **Properties** panel, add the following expression as a measure in the first **Text & image** object:
`='Load Start: ' & vLoadStart`

Label the measure *Load Start*. This expression concatenates text with the time value from the variable.

11. Add the following expression as a measure to the second **Text & image** object:
    ```
    ='Load Complete: ' & vLoadComplete
    ```
    Label the measure *Load Complete.*
12. Add the measure *Reload Time* to the third **Text & image** object with the following expression:
    ```
    ='Reload Time: ' & ReloadTime()
    ```
    Label the measure *Reload Time*.
    ReloadTime() F is a special function that returns the time of the last successful data load. This is the same as **vLoadComplete**.
13. Disable the edit mode in your sheet and observe the results.

Solution



Activity 2: Add an Include statement

Summary:

- Use an Include statement to leverage pre-created, external data load script.
- Discover a way to amalgamate the resulting table into the existing data model, without altering the external script file.

Qlik Sense app: ABC Sales Data

Challenge

Use an **Include** statement to load part of the script from an external file.

- The external file that contains the reusable script is *email.txt*
- Create a new script section named *Email*, following the **SalesPerson** section.
- Add an **Include** statement to create a new table, *EmployeeMail*, using data from the file noted above.
- **RESIDENT LOAD** from the *EmployeeMail* table and create a **JOIN** to *Employees*.
- Drop the original *EmployeeMail* table.
- Check the **Data model viewer**.

Hints

- The **Include** statement is always used inside a dollar-sign expansion syntax.
- The reason using **Include** statements is so common is because it allows code reusability, easier maintenance, and version control options.
- The *Email* field needs to be folded into the *Employees* table without altering the original .txt file. Therefore, a **RESIDENT LOAD** and **JOIN** are used, and the original table is dropped.

Step-by-step guide

1. Use a text editor like Notepad, to examine the file *email.txt*. The code in the file contains a **LOAD...FROM** syntax, geared to using the *employees.xlsx* file, and creates a table called *EmployeeMail*.
   The email is generated automatically using the first letter of the first name, and the first and last letters of the last name. It also ensures that there are no capital letters in the signature, only lowercase letters. Foreign (Swedish) letters, such as å, Å, ä, Ä, ö, and Ö, are substituted by ASCII characters. All of these are handled using nestled **IF** statements.
   Following the signature, @ is appended, followed by the server address. The latter is determined by the employee office.
2. In the **Data load editor**, create a new section between **SalesPerson** and **MasterCalendar**, named *Email*.
3. Using the **Select data** option on the **DataFiles** connection, upload the *email.txt* file from the student collateral, and cancel the **Select file** dialog when upload is complete.

   Qlik Sense on Windows: The *email.txt* file is already available from the **TextFiles** connection.

4. Add an **Include** variable to load the script as an external file:

   ```
   // **** Load emails using include variable ****
   $(Include=[$(vPathFiles)email.txt]);
   ```

   Copy to clipboard

   Qlik Sense on Windows: The *vPathFiles* variable does not exist. Use the *vPathText* instead, which points to the **TextFiles (qse_student)** data connection, where the *email.txt* file resides:

```
// **** Load emails using include variable ****
$(Include=[$(vPathText)email.txt]);
```

Note: Do not leave any spaces before or after the equal sign. If you do not use the square brackets, the file still loads, but the text displays in gray, like a comment.

5. Load the data and verify that the *EmployeeMail* table is now part of your data model. Preview the table to examine its contents.
6. To join *EmployeeMail* to *Employees*, return to the **Email** section within the **Data load editor**.
7. Underneath the **Include** statement, add the following **RESIDENT LOAD** statement from *EmployeeMail*, which joins all the data to *Employees*:

```
// **** Reload emails as a separate table so we can join it ****
LEFT JOIN (Employees)
LOAD *
RESIDENT EmployeeMail;
```

Copy to clipboard

Note: We want to join the new *EmployeeMail* table to *Employees*, without modifying the original, external script. Therefore, a **RESIDENT...LOAD** and **JOIN** are used, and then the original table, *EmployeeMail*, is dropped.

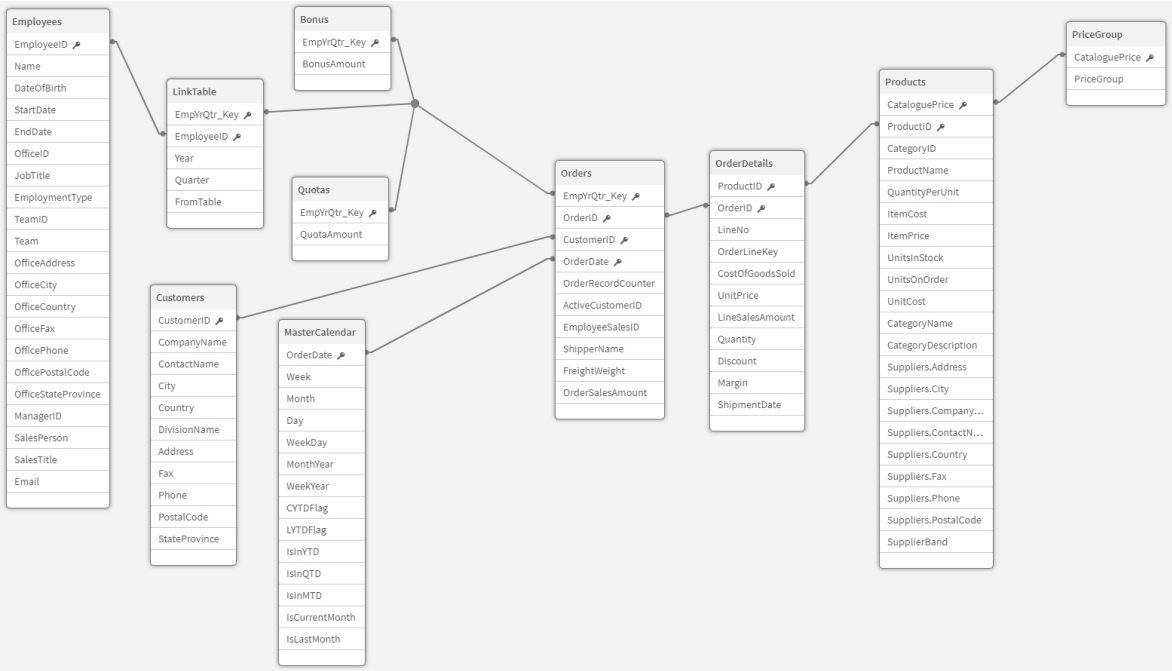
8. Drop the original *EmployeeMail* table by inserting the following script at end of the **Email** section:

```
// **** Drop original email table ****
DROP TABLE EmployeeMail;
```

Copy to clipboard

9. Load the data and check the **Data model viewer** to ensure the *EmployeeMail* table has been joined to *Employees*.

Solution



Activity 3: Use a For loop to load all sheets

Summary:

- Re-engineer the script to load all sheets of Quota data from Excel.
- Create and analyze the resulting data in a table.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, load **all** the sheets from the *Quota.xls* file using a **FOR EACH...NEXT** loop. You can achieve that by doing the following:

- Create a new app sheet named *Quotas* and insert a **Table** with the following dimension fields:
  - *Year*
  - *Quarter*
  - *EmployeeID*
  - *Name*
  - *QuotaAmount*
- Insert a **Filter pane** from the *Year* field.
- When analyzing the table by *year*, notice that only one year contains quota data. You need to load all the tables from the file.

- In the **Quota** section, add a **FOR EACH...NEXT** loop to load data from the tables in the Excel file.
- Use the **Subfield F** function in the *Quotas* table to only keep the numerical part of the *QuotaYear* value.

Hints

- The **vWorkSheet** variable can be used in the **FOR EACH...NEXT** loop to navigate and load all the tables from the Excel file.
- All the existing references to the Excel worksheet name must be replaced with the above variable. The variable must be included in a dollar-sign expansion syntax.

Step-by-step guide

1. Create a new app sheet and name it *Quotas*.
2. In sheet edit mode, insert a **Table** object from the **Assets** panel that fills all but the first three columns on the sheet.
3. Add the following fields as dimensions: *Year*, *Quarter*, *EmployeeID*, *Name*, and *QuotaAmount*.
4. Add the *Year* field as a **Filter pane** in the top left of the sheet and stop editing. Select different years in the filter while checking the table. Notice there is only Quota data for a single year. You need to load the second Excel worksheet to complete the *Quotas* table.
5. Navigate to the **Data load editor**. In the **Quota** section, add the following **FOR** loop before the *QuotasTemp* table:

```
FOR EACH vWorkSheet IN 'Quota 2022','Quota 2023'
```

> **Note:** In the **FOR** loop, define a variable, *vWorkSheet*. Then, use the dollar-sign expansion to evaluate the value of this variable: *$(vWorkSheet)*.

6. In the *QuotasTemp* table, change the **table is** part of the **FROM** clause to connect *[$(vWorkSheet)$]*.

```
table is [$(vWorkSheet)$]);
```

7. Change the *QuotaYear* field to use the variable vWorkSheet. Using the **SUBFIELD** function keeps part of the string in the variable.

```
SUBFIELD('$(vWorkSheet)', ' ', 2) AS QuotaYear,
```

8. Add the **NEXT** statement after the *QuotasTemp* table to close the **FOR** loop. Feel free to indent the inner statements of the **FOR** loop, to aid readability. Your script in the **Quota** section looks as follows:

```
FOR EACH vWorkSheet IN 'Quota 2022','Quota 2023'
    QuotasTemp:
    CROSSTABLE(QuotaQuarter, QuotaAmount, 2)
```

```
    LOAD
        EmployeeID AS QuotaEmployeeID,
    //  SalesPerson,
        SUBFIELD('$(vWorkSheet)', ' ', 2) AS QuotaYear,
        Q1,
        Q2,
        Q3,
        Q4
    FROM [$(vPathFiles)Quota.xls]
    (biff, embedded labels, header is 1 lines, table is
[$(vWorkSheet)$]);
NEXT;
```

Copy to clipboard

Qlik Sense on Windows: The variable used on the Windows versions will be *$(vPathExcel)*, as opposed to *$(vPathFiles)*.

9. Load the data and open the **Quotas** sheet again to review the results. The table now contains values for years 2022 and 2023.

Note: In the above example, the file and sheet names are known and have been hard-coded in the script.
The following **Qlik community link** demonstrates how to dynamically load multiple Excel files from a folder, together with their worksheets and data, when the names are not known.

Solution

Quotas

Year

2022 ✓

2023 ✓

2020

2021

Year	Quarter	EmployeeID	Name	QuotaAmount
2022	Q4	24	Leif Shine	100000
2022	Q4	30	Joan Callins	150000
2022	Q4	32	Rob Carsson	150000
2022	Q4	33	Stefan Lind	40000
2022	Q4	36	Mike Ashkenaz	50000
2022	Q4	44	Ingrid Hendrix	200000
2022	Q4	45	Bob Park	150000
2022	Q4	51	Michael Tyson	200000
2022	Q4	52	Helen Brolin	250000
2022	Q4	55	Tom Lindwall	150000
2022	Q4	161	Valentino Carpachio	150000
2022	Q4	162	Lionel Lazy	150000
2023	Q1	1	Erik Fredriksen	100000
2023	Q1	8	Mike McDonald	75000
2023	Q1	10	Ricardo Gucci	215000
2023	Q1	11	Gwendolyn Cavendish	115000
2023	Q1	12	Fiona Hunter	165000
2023	Q1	13	Evelyn Brown	75000
2023	Q1	15	Helena Du Froid	115000
2023	Q1	16	Frank Roll	90000
2023	Q1	19	Lennart Skoglund	95000
2023	Q1	24	Leif Shine	65000
2023	Q1	30	Joan Callins	45000
2023	Q1	32	Rob Carsson	100000
2023	Q1	33	Stefan Lind	60000
2023	Q1	36	Mike Ashkenaz	50000
2023	Q1	44	Ingrid Hendrix	45000
2023	Q1	45	Bob Park	215000

Activity 4: Debugging a data load script

Summary:

- Use the debugger to place breakpoints and step through a data load script.
- Track and monitor variable values as script execution progresses.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, familiarize yourself with the data load process and debug your script by doing the following:

- Use the debugger to control your data load progress by doing the following:
  - Add two breakpoints in the **Orders** section of your script and use the **Debug** toolbar to control the script execution. The two breakpoints must be added next to the **LOAD** statement for the *Orders* table and next to the *OrderDetails* table name. Observe the load behavior after you execute the script in the debugger.
  - Add breakpoints to the **Include** statement in the **Email** section and the **For Each** statement in the **Quota** section. Observe the debugger behavior as you run and step through script execution.
  - Use the **Variables** section to verify that the value of the **vWorkSheet** variable changes during script execution.

Hints

- The debugger can be found in the **Data load editor**.
- At the end of the activity, all the changes must be reverted back to their initial state.

Step-by-step guide

1. Close any open applications and return to the hub.
2. Open the **ABC Sales Data** app and navigate to the **Data load editor**.
3. Open the script debugger from the **Data load editor** toolbar and analyze the **Variables** section. Underneath **All variables**, there are currently no variables listed. Until one load is executed, there are no variables initialized.
4. Use the **Run** button on the **Debug** toolbar to execute a data load.
5. Observe how variables are initialized during script execution within the **Variables** section.
6. From the list of script sections, select the **Orders** section and set two breakpoints by clicking on the line numbers to the left of the script:

   Q on the line with the **LOAD** for *Orders.*

   Q on the line with the **OrderDetails** table label for *OrderDetails.*
7. Run the script in debug mode again. Notice how the script execution ignores the first breakpoint and stops at the second breakpoint.
8. Run the script again, to complete execution to the end of the script.
9. Use the drop-down in the **Breakpoints** section and select **Delete all** to remove both the breakpoints.
10. Navigate to the **Email** script section.
11. Set a breakpoint on the line with the **Include** statement which loads the employee emails via the external *email.txt* script.
12. Navigate to the **Quota** script section.
13. Set a breakpoint on the line allowing **FOR EACH** to start the loop which loads in the Excel sheets.
14. Use the debug **Run** button to execute the script in debug mode.

15. Processing stops at the **Include** statement breakpoint: Observe how the contents of this external script are displayed within the orange-bordered, statement preview box. Scrolling down the preview box reveals the entire set of statements within the *email.txt* file.
16. Use the **Step** button on the **Debug** panel to proceed one statement.

> **Note:** In reviewing code, it is easier to step through and analyze script lines using breakpoints.

17. Observe that while we can see the content of the external script file within the debugger, moving one step only reflects the script we have in the **Data load editor**. We cannot step through external script, one statement at a time.
18. Use the debug **Run** button to continue script processing.
19. Execution pauses at the next breakpoint, at the start of the **FOR EACH** loop. We want to monitor the **vWorkSheet** variable; scroll down the list of **All variables** and click the **Mark as favorite** icon ï next to *vWorkSheet*.
20. Scroll to the top of the **Variables** section and note that the *vWorkSheet* variable now appears within the **Favorites** area, making it easier to observe as we step through the script. Note that it is currently set to the string *Quota 2023*. This is due to it being the last value *vWorkSheet* was set to during the previous script execution.
21. Press the **Step** button once. Notice that *vWorkSheet* has now been freshly-assigned the new value of *Quota 2022* for this script execution.
22. Use **Step** a couple more times and execution returns to the start of the loop, with *vWorkSheet* now set to *Quota 2023*.
23. Use the debug **Run** button again to complete script execution.
24. Use the drop-down in the **Breakpoints** section again, and **Delete all** the breakpoints in the script.
25. Close the debugger to return to the **Data load editor**.